2017-18 Final

1

File processing system introduces difficulty in accessing data. It also incurs integrity problem. - Place suitable examples to justify these statements.	10
Write down the main responsibilities of a Database Administrator (DBA).	5
What is relational algebra? Briefly outline its major three operations. What is the basic difference between the relational algebra and query language?	10

→ a

→ b

→ c

a) #Difficulty in accessing data

- → inefficient search operation
  - → Ex. search books from Library
- → Limited Query capability
  - → Ex. → Sales management system

Shoyo...

⊟ Integrity Problem

- → Data redundancy and inconsistancy
  - → Ex. Employee record → can have redundancy for multiple employee
  - → any updates may not be work properly
- → Lack of Referential Integrity
  - → Ex. A file system manages course and student data separately.

↳ So there can be create a situation, where a student might have enrolled in a course which doesn't exist.

b → Main responsibilities of DBA

- DB design and modeling
- → DB install and config
- → Backup and Recovery ✓
- → Security management
- → Performance monitoring ✓
- → Capacity Planning
- → Data maintainace
- → Troubleshooting
- → Documentation //
- → DB upgrades

c

Relational Algebra is a formal system used for manipulating relations (tables) in a relational database. It provides a theoretical foundation for relational databases and forms the basis for database query languages. Relational algebra consists of a set of operations that can be performed on relations to retrieve, filter, combine, and manipulate data.

The three major operations in relational algebra are: σ_{cond} relation

1. **Selection (σ):** The selection operation selects specific rows from a relation that satisfy a given condition. It is denoted by σ_{condition}(relation). For example, σ_{Age > 25}(Employees) selects rows from the "Employees" relation where the "Age" attribute is greater than 25.
2. **Projection (π):** The projection operation selects specific columns (attributes) from a relation, discarding the rest. It is denoted by π_{attributes}(relation). For example, π_{Name, Salary}(Employees) selects only the "Name" and "Salary" columns from the "Employees" relation.
3. **Join (⋈):** The join operation combines tuples from two or more relations based on a common attribute to create a new relation. It is denoted by R₁ ⋈_{condition} R₂. For example, Employees ⋈_{DepartmentID} Departments joins the "Employees" and "Departments" relations based on the "DepartmentID" attribute, combining relevant data from both relations.

$\pi_{attr\ list}$

$R_1 \bowtie_{cond} R_2$

Basic Differences Between Relational Algebra and Query Languages:

1. **Formality and Theoretical Foundation:**
   - **Relational Algebra:** Relational algebra is a formal, mathematical system with well-defined operations used to manipulate relations. It serves as a theoretical foundation for relational databases.
   - **Query Languages (e.g., SQL):** Query languages like SQL (Structured Query Language) are more user-friendly and provide a higher level of abstraction. SQL queries are written in a human-readable syntax and are translated by the database management system into relational algebraic operations for execution.
2. **Level of Abstraction:**
   - **Relational Algebra:** Relational algebra operates at a lower level of abstraction, dealing directly with relations and their operations.
   - **Query Languages:** Query languages provide a higher level of abstraction, allowing users to express queries and operations in a more intuitive and natural way without having to worry about the underlying relational algebraic operations.
3. **Ease of Use and Expressiveness:**
   - **Relational Algebra:** Relational algebraic expressions can be complex and may require a deep understanding of the formal system to write advanced queries.
   - **Query Languages:** Query languages are designed to be user-friendly, allowing users to express complex queries more intuitively and with greater ease. SQL, for example, provides a wide range of expressive capabilities to handle various data manipulation tasks.

In summary, relational algebra provides a formal and foundational approach for working with relational databases, whereas query languages like SQL offer a more user-friendly and expressive interface for interacting with databases, abstracting away the complexities of the underlying relational algebraic operations.

2(a)

3×4

Consider the following database design:

employee (person name, street, city)
works (person name, company name, salary)
company (company name, city)

Give expressions both in the relational algebra and standard SQL to express each of the following queries:

i. Find the names of all employees who live in city "Dhaka".
ii. Find the names of all employees whose salary is greater than $60000.
iii. Find the names of all employees who live in "Dhaka" and whose salary is greater than $60000.

1. → $\Pi_{name}(\sigma_{city = "Dhaka"}(employee))$

→ select name
from employee
where city = 'Dhaka';

2. $\Pi_{name}(\sigma_{salary>'6500'}(works))$

```
select name
from works
where salary>6500;
```

3 $\Pi_{name}(\sigma_{city='Dhaka' \wedge salary>6000}$

(employee $\bowtie_{name=name}$ works)

```
select e.name
from employee e
JOIN works w ON e.name=w.name
where e.city ___ AND w.salary___
```

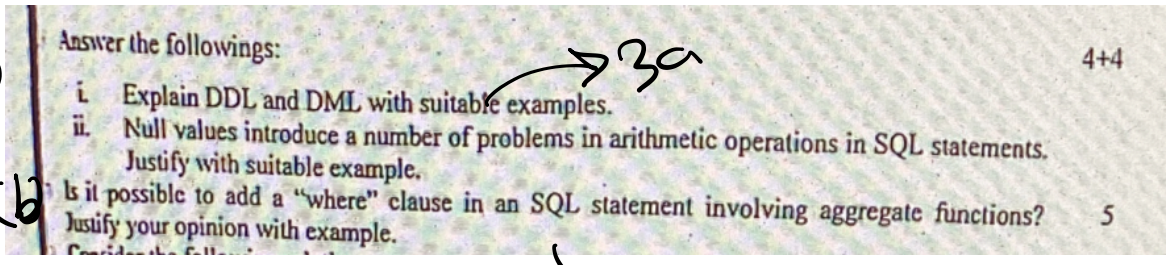
2(b)+(c)

What is the basic difference between DDL and DML? Explain with example. 5
What is the difference between inner join and outer join? Explain left outer join and right outer join with suitable example data. 8

Solve → JJK!?? Slide → Direct ans

3(a) i → DDL, DML,

(a)



Answer the followings: 4+4

i. Explain DDL and DML with suitable examples. → 3a

ii. Null values introduce a number of problems in arithmetic operations in SQL statements. Justify with suitable example.

(b)(b) Is it possible to add a "where" clause in an SQL statement involving aggregate functions? Justify your opinion with example. 5

(ii) → Addition / subs. with NULL → Result NULL

↳ Division & Multiplication with NULL ↑

↳ give examples

→

Handling NULL Values:

To handle these situations, you can use the `COALESCE()` or `ISNULL()` functions to replace NULL values with a default value before performing arithmetic operations. For instance:

Query 4:

sql Copy code

```
SELECT OrderID, COALESCE(TotalAmount, 0) + 50 AS NewAmount
FROM Orders;
```

In this case, `COALESCE(TotalAmount, 0)` replaces NULL with 0, ensuring predictable arithmetic results.

By being aware of NULL values and using appropriate handling mechanisms, you can avoid unexpected outcomes in arithmetic operations involving SQL data.

-

3(b)

Yes, it is possible to use a `WHERE` clause in SQL queries involving aggregate functions. The `WHERE` clause allows you to filter the rows before applying aggregate functions. This can be useful when you want to calculate aggregates only for specific subsets of data. Here's an example to illustrate this:

Consider a table `Sales` with columns `ProductID`, `Category`, and `Amount`:

ProductID	Category	Amount
1	A	100
2	B	150
3	A	200
4	C	120
5	A	180

Example 1: Calculating Average Amount for Category 'A'

To calculate the average `Amount` for products in Category 'A', you can use the `AVG()` aggregate function with a `WHERE` clause:

sql Copy code

```sql
SELECT AVG(Amount) AS AverageAmount
FROM Sales
WHERE Category = 'A';
```

The `WHERE` clause filters the rows where `Category` is 'A', and then the `AVG()` function calculates the average `Amount` for these filtered rows.

Example 2: Finding Maximum Amount for Products with Amount Greater Than 150

To find the maximum `Amount` for products where `Amount` is greater than 150, you can use the `MAX()` aggregate function with a `WHERE` clause:

sql Copy code

```sql
SELECT MAX(Amount) AS MaximumAmount
FROM Sales
WHERE Amount > 150;
```

In this case, the `WHERE` clause filters the rows where `Amount` is greater than 150, and then the `MAX()` function calculates the maximum `Amount` for these filtered rows.

Using the `WHERE` clause with aggregate functions allows you to perform calculations on specific subsets of data based on the conditions specified in the `WHERE` clause. This enables you to derive meaningful insights from your data based on specific criteria.

3(c)

Consider the following relations: 2×6

(Note: ID is the primary key of each entity. x(FK[r]) indicates x a foreign key referencing entity r)

persons(ID, Name, DOB, Address)
schools(ID,Name, Establish_Year)
companies(ID, Name, Location)
students(ID,Person_ID (FK[persons], gpa, school_ID(FK[schools]))
emp(ID, Person_ID(FK[persons]), Salary)

Write the following SQLs:

i. List the person Name, ID and Address according to their age (i.e. Oldest ... first)
ii. List each student's information as following:
Student ID, Student Name, Name of School, gpa
iii. List the school's summary as : School Name, Total Students, average gpa
iv. List top 5 schools based on the average gpa (as obtained in iii)
v. List employees name, his/her company name, salary
vi. Update each employee salary by 20% for those who currently get less than the ... salary of his/her company's employees salary

Sol

i. List the person Name, ID, and Address according to their age (oldest first):

```sql
SELECT ID, Nome AS Name, Address
FROM person
ORDER BY DOB ASC;
```

ii. List each student's information as following: Student ID, Student Name, Name of School, GPA:

```sql
SELECT s.ID AS StudentID, p.Nome AS StudentName, sch.Name
FROM students s
JOIN person p ON s.PersonID = p.ID
JOIN schools sch ON s.school_ID = sch.ID;
```

iii. List the school's summary as: School Name, Total Students, Average GPA:

```sql
SELECT sch.Name AS SchoolName, COUNT(s.ID) AS TotalStuden
FROM students s
JOIN schools sch ON s.school_ID = sch.ID
GROUP BY sch.Name;
```

iv. List the top 5 schools based on the average GPA (as obtained in iii):

```sql
SELECT SchoolName, TotalStudents, AverageGPA
FROM (
    SELECT sch.Name AS SchoolName, COUNT(s.ID) AS TotalSt
    FROM students s
    JOIN schools sch ON s.school_ID = sch.ID
    GROUP BY sch.Name
    ORDER BY AverageGPA DESC
    LIMIT 5
) AS TopSchools;
```

v. List employees' names, their company names, and salaries:

```sql
SELECT e.ID AS EmployeeID, p.Nome AS EmployeeName, c.Name
FROM emp e
JOIN person p ON e.PersonID = p.ID
JOIN companies c ON e.company_ID = c.ID;
```

vi. Update each employee's salary by 20% for those who currently get less than the salary of his/her company's employees' salary:

```sql
UPDATE emp e
SET Salary = Salary * 1.20
WHERE Salary < ALL (
    SELECT Salary
    FROM emp e2
    WHERE e2.company_ID = e.company_ID
);
```

4 a) Define

1. Super Key:

A **super key** is a set of one or more attributes (columns) in a table that can uniquely identify a record (row). Any set of attributes that includes a super key is also a super key. In other words, a super key is a combination of columns that uniquely identifies a record in a table.

Example:

Consider a table `Employees` with columns `EmployeeID`, `Email`, `SSN` (Social Security Number), and `Phone`.

- `{EmployeeID}` and `{Email, Phone}` are both super keys because they can uniquely identify each employee in the table:

2. Candidate Key:

A **candidate key** is a minimal super key, meaning it is a super key with the minimum number of attributes necessary to uniquely identify a record. In other words, it is an irreducible super key - if you remove any attribute from a candidate key, it will no longer be unique.

Example:

In the `Employees` table, `EmployeeID` and `Email` are both candidate keys because they are minimal and can uniquely identify each employee. If you remove any attribute from these keys, they would no longer be unique.

3. Primary Key:

A **primary key** is a specific candidate key chosen to uniquely identify each record in a table. It must be unique for each record and cannot contain NULL values. A table can have only one primary key.

Example:

In the `Employees` table, `EmployeeID` can be chosen as the primary key because it is unique for each employee and does not contain NULL values. Once `EmployeeID` is declared as the primary key, it uniquely identifies each record in the table.

In summary, a super key is any combination of attributes that uniquely identifies a record, a candidate key is a minimal super key, and a primary key is a chosen candidate key used to uniquely identify records in a table.

4(c) View ৩ Prev. yr Q+২ আসে

d)

d) i. What is cardinality? How do you ensure many-many cardinality? Use example explain.
ii. Differentiate between Cartesian product and natural join.
iii. *"Natural join removes meaningless records."* - Justify with suitable example.

d) i)

Cardinality in the context of databases refers to the number of unique values in a column of a table. It defines the relationship between tables. In a many-many cardinality, each record in the first table can be associated with multiple records in the second table, and vice versa.

To ensure a many-many cardinality between two entities in a relational database, you create an **association table** (also known as a **junction table** or a **linking table**). This table holds the foreign keys of the two entities, forming a composite primary key. It allows multiple records in each table to be associated with multiple records in the other table.

Example:
Consider two entities: `Students` and `Courses`. Each student can enroll in multiple courses, and each course can have multiple students. To represent this many-many relationship, you create an `Enrollments` table:

sql — Copy code

```sql
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(255)
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(255)
);

CREATE TABLE Enrollments (
    StudentID INT,
    CourseID INT,
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

ii. Cartesian Product vs. Natural Join:

- **Cartesian Product:** A Cartesian product of two tables returns all possible combinations of rows from the two tables. For example, if Table A has m rows and Table B has n rows, their Cartesian product will have m * n rows.

  Example:

  sql — Copy code

  ```sql
  SELECT * FROM TableA, TableB;
  ```

- **Natural Join:** A natural join between two tables combines rows based on columns with the same name. It automatically matches columns with identical names and returns the result.

  Example:

  sql — Copy code

  ```sql
  SELECT * FROM TableA NATURAL JOIN TableB;
  ```

iii. "Natural Join Removes Meaningless Records" - Justification:

Natural join removes meaningless records by focusing on meaningful relationships between tables. It matches rows based on common attributes and discards rows where the matching attribute values are not meaningful in the context of the relationship between the tables.

Example:
Consider two tables, `Employees` and `Projects`, where both tables have a `DepartmentID` column. If you perform a natural join on these tables, it will match employees with projects in the same department. Rows where the `DepartmentID` values do not match (or where they match but are not meaningful) will be discarded.

sql — Copy code

```sql
SELECT * FROM Employees NATURAL JOIN Projects;
```

In this scenario, the natural join ensures that only employees and projects within the same department are combined, removing any meaningless combinations that do not reflect actual relationships within the organization.